

5교시: 실패 분석 라이프사이클 - reproduce, observe, hypothesize, fix, recheck, prevent

수업 목표

- 실패 분석을 6단계로 기록한다.
- symptom과 root cause candidate를 구분한다.
- 없는 파일 요청으로 404를 만들고 RCA를 작성한다.

50분 흐름

Time	Activity
0-5분	README 재현성 checklist 확인
5-15분	RCA와 symptom/cause 차이 설명
15-30분	404 실패 재현과 log 관찰
30-40분	fix/recheck/prevent 기록
40-50분	관찰 가능성 signal로 연결

0-5분 README 재현성 checklist 확인

- 진행: README 재현성 checklist 확인
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

상세 설명

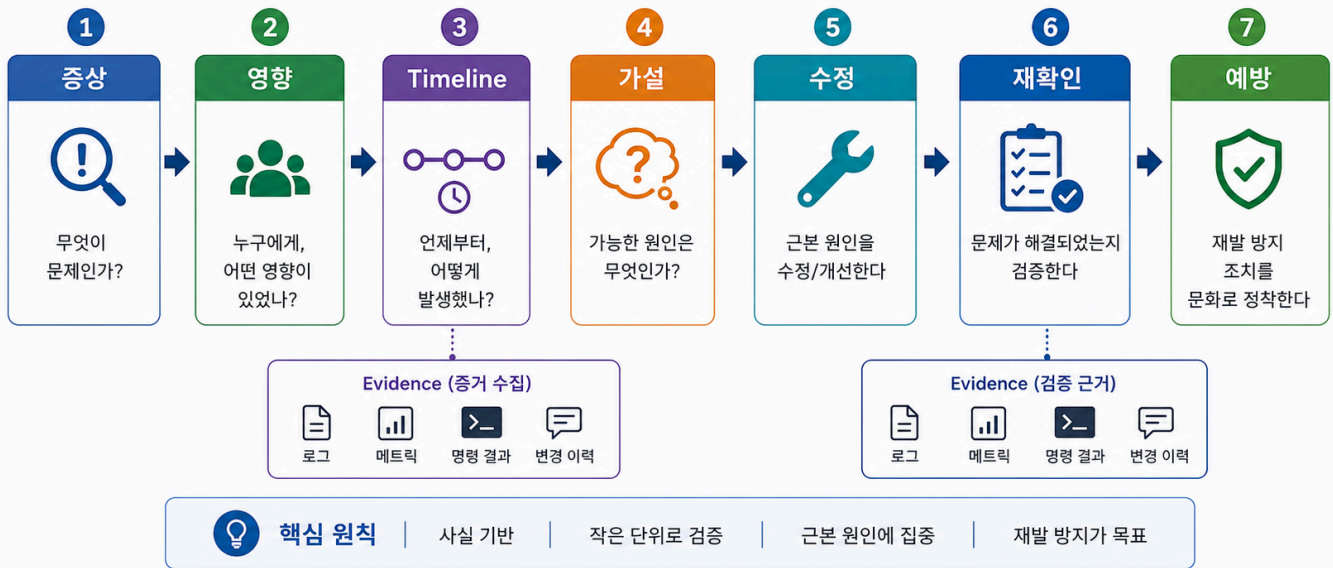
RCA는 누군가를 탓하는 문서가 아니라 같은 실패를 줄이기 위한 학습 기록이다. 첫 단계는 실패를 재현하는 것이다. 재현할 수 없으면 관찰도 비교도 어렵다. 다음으로 log, status, command output을 관찰하고, 원인 후보를 세운다. fix를 적용한 뒤 같은 check로 다시 확인하고, 마지막으로 재발 방지 방법을 남긴다.

오늘은 없는 파일을 요청해 404를 만든다. 404는 서버가 죽었다는 뜻이 아니라 server가 요청을 받았지만 해당 path의 resource를 찾지 못했다는 뜻이다. 이 차이를 이해하면 connection refused, 404, 500을 섞어 말하지 않게 된다.

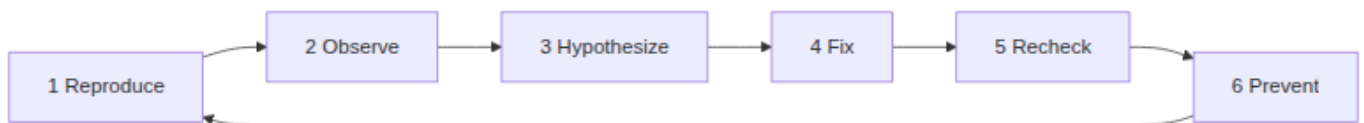
Visual 1: RCA lifecycle

RCA 기본 흐름

문제의 근본 원인을 찾고, 재발을 막는 체계적 과정



이 이미지는 RCA를 책임 추궁이 아니라 관찰 가능한 증거로 문제를 좁히는 절차로 다룬다. 특히 Timeline 과 Recheck 를 분리해 “수정했다”와 “수정이 확인됐다”를 구분한다.



RCA는 한 번 쓰고 끝나는 보고서가 아니라 같은 실패를 줄이는 반복 구조다. 오늘은 작은 404로 이 흐름을 연습한다.

5-15분 RCA와 symptom/cause 차이 설명

- 진행: RCA와 symptom/cause 차이 설명
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

Visual 2: Status별 첫 판단

Symptom	process 상태 추정	먼저 볼 evidence
connection refused	서버 process가 없거나 port가 다름	server terminal, port
404	process는 응답함, path/resource 문제 가능	URL path, ls 결과, request log
500	process는 응답함, 내부 처리 실패 가능	error log, recent change
browser blank	HTML 내용 또는 wrong file 가능	curl, cat index.html

상태 코드는 정답이 아니라 출발점이다. "어디가 고장났을 가능성이 높은가"를 좁히는 evidence로 사용한다.

15-30분 404 실패 재현과 log 관찰

- 진행: 404 실패 재현과 log 관찰

- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

RCA 템플릿

Step	Record
Reproduce	
Observe	
Hypothesize	
Fix	
Recheck	
Prevent	

명령 절차

서버가 실행 중인 상태에서 새 terminal을 연다.

```
curl -I http://localhost:8000/no-such-file.html
```

수정 방법은 새 기능 구현이 아니라 올바른 existing path로 확인하는 것이다.

```
curl -I http://localhost:8000/index.html
```

30-40분 fix/recheck/prevent 기록

- 진행: fix/recheck/prevent 기록
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

확인 질문

- 이번 실패의 symptom은 무엇인가?
- root cause candidate는 process, port, file path 중 어디에 가까운가?
- recheck에 같은 명령을 다시 써야 하는 이유는 무엇인가?

다음 주차 매핑

Docker에서는 wrong file path가 image build context 문제로 나타날 수 있고, Kubernetes에서는 wrong path가 readiness 실패로 보일 수 있다. AWS 배포에서는 ALB target health와 application status를 분리해서 봐야 한다.

예상 결과

- 없는 파일은 보통 404 status를 반환한다.
- 서버 terminal에는 /no-such-file.html 요청 log가 남는다.
- /index.html 요청은 200 status를 반환해야 한다.

흔한 오해

오해	교정
404는 서버 process가 죽은 것이다.	server가 응답했으므로 process는 살아 있다. path/resource 문제다.
RCA는 큰 장애 때만 쓴다.	작은 실패에서 연습해야 큰 장애 때 구조적으로 기록할 수 있다.
fix만 하면 기록은 필요 없다.	prevent가 없으면 같은 실수가 반복된다.

40-50분 관찰 가능성 signal로 연결

- 진행: 관찰 가능성 signal로 연결
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

실습 Evidence

Evidence	Value
failing URL	
failing status	
server log excerpt	
fixed check URL	
fixed status	
prevention note	

학술 근거와 DevOps insight

SRE와 incident management에서는 증상, 영향, 원인, 조치, 재발 방지를 분리해 기록한다. 작은 404 분석도 같은 구조를 사용한다. 현업에서 좋은 RCA는 정답 맞추기가 아니라 evidence를 근거로 원인 후보를 좁히는 과정이다.

평가 기준

기준	2점 evidence
50분 참여	시간 흐름에 맞춰 설명, 활동, 산출물 작성에 참여했다.
증거 산출	수업에서 요구한 note, command, table, blocker 중 해당 산출물을 구체적으로 남겼다.
전이 연결	오늘 개념이 Week2~6 기술 또는 자기 산출물과 어떻게 연결되는지 한 문장 이상 설명했다.

공식/학술 근거 링크

- Google SRE Book: Postmortem Culture, <https://sre.google/sre-book/postmortem-culture/> - RCA를 blame이 아니라 학습 가능한 evidence로 남기는 기준이다.
- RFC 9110: HTTP Semantics, <https://datatracker.ietf.org/doc/html/rfc9110> - 404와 같은 status code를 원인 후보 분리에 사용하는 공식 기준이다.
- Google SRE Book: Introduction, <https://sre.google/sre-book/introduction/> - monitoring, emergency response, change management를 운영 책임으로 보는 근거다.